

Deep Learning

Project 2 report

Speech Commands Classification with Transformers

Bogumiła Okrojek

Katsiaryna Bokhan

May 2026

Abstract

This project presents an empirical study of attention-based architectures for speech command recognition using the TensorFlow Speech Recognition Challenge dataset. Three core architectures are evaluated: a lightweight Conformer, a standard Audio Spectrogram Transformer (AST), and a pretrained Wav2Vec2 model fine-tuned for the keyword spotting task. We consider two primary methods for transforming raw audio signals into 2D representations: Mel-scale spectrograms and Mel-frequency cepstral coefficients (MFCC). We investigate the effect of these feature extraction configurations, specifically the number of MFCC coefficients, Mel-filter banks, and hop length, as well as transformer-specific hyperparameters including the number of attention heads, patch size, and embedding dimension. Furthermore, we analyze the impact of data balancing strategies through under- and over-sampling techniques for the non-target classes “silence” and “unknown”. Additionally, we perform a comparative study of two AST variations – a standard single-head classifier and a proposed hierarchical dual-head architecture designed to decouple “silence” detection from command identification.

Contents

1	Problem description	3
2	Theoretical introduction	3
2.1	Audio to spectrogram transformation	3
2.2	Transformers and self-attention mechanism	4
2.3	Conformer: convolutions + attention	5
2.4	Double-head audio spectrogram transformer	5
3	Literature Review	5
3.1	Audio Feature Extraction: Spectrograms and MFCCs	5
3.2	Neural Networks for Speech and Audio Classification	6
3.3	Transformers and Self-Attention for Speech	6
3.4	Conformer: Combining Convolution and Attention	6
4	Experiments	8
4.1	Input representation experiment	8
4.2	Transformer parameters experiment	8
4.3	Analysis of under/oversampling strategies for silence/unknown classes	9
4.4	Analysis of single-head and double-head AST architectures	9
4.5	Different architecture comparison	10
5	Results	11
5.1	Input representation experiment	11
5.2	Transformer parameters experiment	12
5.3	Analysis of under/oversampling strategies for silence/unknown classes	14
5.4	Analysis of single-head and double-head AST architectures	17
5.5	Different architecture experiment	17
6	Conclusion	18
7	Instruction of the application (reproducibility)	19
7.1	Environment setup	19
7.2	Dataset preparation	20
7.3	Running experiments	20
8	AI Usage Declaration	20
A	Input representation experiment	22
B	Analysis of under/oversampling strategies for silence/unknown classes	25
C	Analysis of single-head and double-head AST architectures	26
D	Analysis of different model architectures	27

1 Problem description

Keyword spotting is a critical task in speech processing, aimed at identifying specific utterances within an audio stream.

The objective of this project is to evaluate and compare different neural network architectures in a multiclass setting using the *Speech Commands* dataset from the *TensorFlow Speech Recognition Challenge* [4]. The dataset consists of over 150,000 one-second audio clips distributed across 12 target categories: 10 core commands (*yes, no, up, down, left, right, on, off, stop, and go*), an *unknown* class, and a *silence* class.

The task is to train models capable of correctly assigning each audio segment to one of the predefined classes. In some of our experiments, we vary the proportions of the “unknown” and “silence” classes relative to the core commands to investigate their impact on the models’ generalization capabilities. To ensure an objective evaluation under these varying conditions, **balanced accuracy** is employed as the primary performance metric, supplemented by the analysis of **confusion matrices** to track inter-class misclassifications.

The study focuses on analysis how different factors influence model performance. In particular, the following aspects are investigated:

- the choice of transformer-based architectures;
- audio-to-image conversion techniques (MFCC vs. Mel Spectrograms) and their associated hyperparameters;
- internal transformer hyperparameters, including embedding dimension, number of attention heads, and patch size;
- the impact of varying the number of samples in the “silence” and “unknown” classes on training quality;
- the effectiveness of splitting the task into two stages – first distinguishing “silence” from speech and then identifying the specific command – compared to the standard method where all 12 classes are classified simultaneously.

2 Theoretical introduction

2.1 Audio to spectrogram transformation

The modern standard for acoustic signal processing in deep learning involves transforming 1D raw audio waveforms into 2D time-frequency representations known as spectrograms. This transformation enables models to process speech data by treating it as a structured image-like object. In this project, we analyze two primary methods for feature extraction:

Mel-spectrograms This representation maps acoustic energy onto the Mel scale, which approximates human auditory perception [15] by providing finer resolution at lower frequencies where speech information is most dense. The dimensions of the resulting feature map are governed by two critical hyperparameters:

- **N_MELS:** This parameter defines the frequency resolution, effectively acting as the “height” of the resulting spectrogram. It determines how many frequency bins the audio is divided into; a higher value provides a more detailed representation of the pitch but increases the input dimensionality.
- **HOP_LENGTH:** This parameter governs the temporal resolution, acting as the “width” of the image. It specifies the step size between successive analysis windows; a smaller HOP_LENGTH captures more rapid changes in speech (such as short consonants) by creating a denser sequence of frames.

Mel-frequency cepstral coefficients (MFCC) MFCCs provide a compressed representation of the spectral envelope by applying a discrete cosine transform to the log-Mel-spectrogram. This method is highly effective for keyword spotting as it isolates phonetically relevant information – such as the vocal tract shape required to pronounce a specific command – while filtering out redundant acoustic noise [3].

STFT parameters Spectrogram-based representations rely on the Short-Time Fourier Transform (STFT). For the 1-second audio clips sampled at 16,000 Hz, we use a window size of `N_FFT = 1024`, which determines the frequency resolution of the analysis. Larger window sizes improve frequency resolution but reduce temporal precision.

Raw waveform preprocessing In approaches based on raw audio input, the signal is preserved in its original 1D temporal form without applying explicit time–frequency transformations. To ensure consistency across samples, the waveform is typically resampled to a fixed sampling rate (commonly 16 kHz) and converted to a single channel. Additionally, all recordings are standardized to a uniform duration through truncation or zero-padding, resulting in fixed-length input vectors. Amplitude normalization is applied to control the dynamic range of the signal and improve numerical stability. These preprocessing steps provide a consistent and well-conditioned input representation while retaining the full temporal resolution of the original audio signal.

2.2 Transformers and self-attention mechanism

Transformers, introduced in [16], are a class of neural architectures based entirely on attention mechanisms, eliminating the need for recurrent or convolutional operations. Their core component, **self-attention**, enables the model to capture dependencies between all elements of an input sequence regardless of their distance, making them particularly well-suited for sequential data such as speech signals [14].

Given an input sequence of embeddings, self-attention computes three representations for each element: queries (Q), keys (K), and values (V), obtained through learned linear projections. The attention mechanism then evaluates pairwise interactions between elements by computing scaled dot-products:

$$\text{Attention}(Q, K, V) = \text{softmax} \left(\frac{QK^\top}{\sqrt{d_k}} \right) V,$$

where d_k is the dimensionality of the key vectors. This operation produces a weighted combination of all elements in the sequence, allowing the model to dynamically focus on the most relevant parts of the input.

To increase representational capacity, Transformers employ **multi-head attention**, where multiple attention operations are performed in parallel, each learning different types of relationships. Their outputs are concatenated and projected back into the embedding space.

A standard Transformer encoder layer consists of two main components applied with residual connections and normalization:

1. **Multi-head self-attention:** captures global dependencies across the sequence.
2. **Feed-forward network:** a position-wise multilayer perceptron (typically with expansion factor $\times 4$ and non-linear activation such as GELU).

Since Transformers lack an inherent notion of sequence order, **positional encoding** is added to the input embeddings to inject information about element positions. This can be implemented using fixed sinusoidal functions or learned embeddings [16].

In the context of audio processing, Transformer-based models typically operate on sequences of feature vectors derived from spectrogram patches, treating them analogously to tokens in natural language or patches in vision models [5]. This formulation enables the model to jointly model temporal and frequency relationships through global attention mechanisms.

In this project We employ a standard Transformer encoder-based architecture inspired by the Audio Spectrogram Transformer (AST) framework. The input spectrogram is first divided into overlapping patches using a convolutional projection, which maps local time–frequency regions into embedding vectors. A learnable classification token is appended to the sequence, and positional embeddings are added to preserve spatial structure. The resulting sequence is processed by a stack of Transformer encoder layers, and classification is performed using the representation of the classification token after the final layer normalization stage.

2.3 Conformer: convolutions + attention

The Conformer [7] combines multi-head self-attention with depthwise separable convolution inside each encoder block, addressing the complementary strengths of the two paradigms: attention captures long-range dependencies across the sequence, while convolution extracts local acoustic patterns efficiently.

Each Conformer block follows the **Macaron** structure – a feed-forward module is placed both before and after the attention-convolution core, each contributing with a half-step residual ($\times 0.5$). Concretely, a single block applies the following operations in order, with residual connections at each stage:

1. **Feed-forward module** (half-scaled): LayerNorm $\rightarrow$ Linear (expand $\times 4$) $\rightarrow$ SiLU $\rightarrow$ Dropout $\rightarrow$ Linear $\rightarrow$ Dropout, added with weight 0.5.
2. **Multi-head self-attention module**: LayerNorm $\rightarrow$ MultiheadAttention $\rightarrow$ Dropout.
3. **Convolution module**: LayerNorm $\rightarrow$ pointwise Conv1d (expand $\times 2$) $\rightarrow$ GLU $\rightarrow$ depthwise Conv1d $\rightarrow$ BatchNorm $\rightarrow$ SiLU $\rightarrow$ pointwise Conv1d $\rightarrow$ Dropout.
4. **Feed-forward module** (half-scaled): identical to step 1.
5. **LayerNorm** applied to the block output.

The input spectrogram is first divided into patches via a 2D convolutional projection (**PatchEmbed**), producing a sequence of patch embeddings to which learned positional embeddings are added. The sequence is then passed through L stacked Conformer blocks. Classification is performed by average pooling over the full patch sequence followed by a linear head.

In this project we employ a *MiniConformer* – a custom lightweight variant with `num_heads` = 4, `depth` = 4, and a depthwise convolution kernel of size 7 - to keep training feasible under the computational constraints of the project.

2.4 Double-head audio spectrogram transformer

The standard AST uses a single linear head applied to the [CLS] token to classify all 12 categories simultaneously. We propose a hierarchical variant that decomposes this into two parallel sub-tasks, motivated by the structural asymmetry of the label space: *silence* is acoustically distinct from all speech commands, and forcing a single head to resolve both the *silence*-vs.-speech boundary and within-speech distinctions simultaneously is an unnecessary conflation of two different problems.

The encoder is identical to the baseline AST. Two independent linear heads are attached to the shared [CLS] representation:

- **Silence head** – a binary classifier that distinguishes *voice* from *silence*.
- **Command head** – an 11-class classifier covering the 10 core commands and the *unknown* class.

During training, both heads receive gradients simultaneously, so the shared encoder benefits from two complementary supervision signals. At inference, the two outputs are combined hierarchically: if the silence head predicts *silence*, that label is assigned; otherwise the command head determines the final class.

3 Literature Review

3.1 Audio Feature Extraction: Spectrograms and MFCCs

The transformation of raw audio waveforms into two-dimensional time-frequency representations is a foundational step in modern speech processing. The dominant paradigm involves computing a Short-Time Fourier Transform (STFT) to obtain a spectrogram, which captures how the frequency content of a signal evolves over time [1].

A widely used refinement is the **Mel-scale spectrogram**, which maps frequency bins onto the perceptually motivated Mel scale [15]. This scale provides higher resolution at lower frequencies where most phonetic information is concentrated, and compresses higher frequencies accordingly, making it well-suited for speech tasks as it aligns more closely with human auditory perception than a linear frequency axis.

Mel-Frequency Cepstral Coefficients (MFCCs) take this a step further by applying a Discrete Cosine Transform (DCT) to the log-Mel spectrogram, yielding a compact set of coefficients that capture the spectral envelope of the signal [13]. MFCCs have been a standard feature in speech recognition for decades, owing to their ability to isolate phonetically relevant information while suppressing noise [9]. Davis and Mermelstein (1980) demonstrated their effectiveness for monosyllabic word recognition, a result that has been widely replicated in keyword spotting settings [3].

3.2 Neural Networks for Speech and Audio Classification

The application of deep neural networks to audio and speech classification has been evolving through decades. Early work relied on hand-crafted features fed into shallow classifiers such as Gaussian Mixture Models or Hidden Markov Models. The breakthrough of deep learning, particularly the demonstration of deep neural networks for acoustic modelling by Hinton et al. [9], triggered a rapid transition toward end-to-end learned representations.

Convolutional Neural Networks (CNNs) emerged as the dominant architecture for spectrogram-based audio classification. By treating the spectrogram as an image, CNNs can exploit local structure in time and frequency through shared convolutional filters [10]. LeCun and Bengio established the theoretical basis for applying convolutions to images and time-series data, and subsequent work extended these insights to speech. Architectures such as ResNet [8], originally developed for image recognition, proved highly effective when adapted to audio spectrograms, achieving strong results on keyword spotting benchmarks.

For the specific task of **keyword spotting**, Warden established the benchmark of spoken words from one-second audio clips [17]. Recently a systematic study of deep learning approaches for simple speech command recognition was conducted, confirming the effectiveness of convolutional architectures while noting their limitations in capturing global context [14].

A key limitation of CNNs is their reliance on local receptive fields: while they efficiently capture local patterns such as formants or transients, they struggle to model long-range dependencies across the full temporal extent of an utterance without stacking many layers. This motivated the introduction of recurrent architectures and, ultimately, the shift toward attention-based models.

3.3 Transformers and Self-Attention for Speech

The Transformer architecture replaced recurrent sequence processing with a mechanism of scaled dot-product self-attention [16]. Each token in a sequence computes attention scores over all other tokens, allowing the model to capture dependencies regardless of their distance. The use of **multi-head attention** further enables the model to simultaneously attend to information from different representation subspaces, a property that proved essential for capturing the diverse acoustic cues present in speech.

The success of Transformers in natural language processing spurred their rapid adoption in audio and speech. A key milestone was **wav2vec 2.0** [2], a self-supervised model that learns speech representations directly from raw audio. It uses a convolutional feature encoder followed by a Transformer context network and is pre-trained on large amounts of unlabelled audio via a contrastive objective over masked latent representations. When fine-tuned on labelled data, wav2vec 2.0 achieves state-of-the-art results with very little supervision, demonstrating the power of transfer learning for speech.

The **Audio Spectrogram Transformer (AST)** [5] purely attention-based, convolution-free model for audio classification. AST divides the input spectrogram into fixed-size overlapping patches, linearly projects each patch into an embedding, and processes the resulting sequence through a standard Vision Transformer encoder. A follow-up, SSAST [6], extended this with self-supervised pre-training using masked spectrogram patch modelling, significantly improving performance in low-data regimes.

3.4 Conformer: Combining Convolution and Attention

While pure Transformer models excel at capturing global context, they inherently lack the local inductive bias of convolutional architectures. For speech recognition, where fine-grained local patterns such as phoneme boundaries and formant transitions are as important as long-range structure, this trade-off is particularly significant.

The **Conformer** architecture [7] was designed to address this limitation by integrating multi-head self-attention and depthwise separable convolution within each encoder block. The design follows a Macaron structure: two half-step feed-forward modules sandwich an attention module and a convolution module, with residual connections at each stage. This arrangement allows the model to simultaneously

exploit local acoustic patterns through convolution and long-range dependencies through attention, in a parameter-efficient manner.

Subsequent work validated the Conformer design across a range of tasks beyond speech recognition, e.g. demonstrating their effectiveness for environmental sound classification [11]. The modular structure of the Conformer block also facilitates lightweight variants that reduce computational cost while retaining the complementary local-global modelling capability, making the architecture practical for resource-constrained settings.

4 Experiments

All trained neural networks based on AST and Conformer architectures were trained for 30–50 epochs (depending on the specific experiment) using the Early Stopping technique with a patience of 5 epochs. The Wav2Vec2 model, which was fine-tuned for this task, was trained for 10 epochs with a patience of 5, as the computational requirements and training time per epoch for this architecture were higher, and the model tends to overfit quickly. Furthermore, data augmentation was consistently applied across all experiments using the *SpecAugment* method [12], (`freq_mask_param=10`, `time_mask_param=8`). To ensure comparability across experiments, all hyperparameters not under investigation in a given experiment were kept fixed at their default values throughout the study.

4.1 Input representation experiment

In these experiments, we investigate the impact of various feature extraction methods and their parameters on the classification performance of the MiniConformer architecture. An analysis is conducted between two widely used audio representations:

- **Mel-spectrograms:** A frequency-domain visualization mapped to the Mel scale to reflect human auditory perception.
- **MFCC (Mel-Frequency Cepstral Coefficients):** Coefficients representing the short-term power spectrum of sound, commonly utilized in speech recognition tasks.

Common Features Across all experiments, several parameters were kept constant: the sampling rate (`SAMPLE_RATE` = 16,000 Hz), the Fast Fourier Transform window size (`N_FFT` = 1024), and the number of Mel filters (`N_MELS` = 128) for the intermediate spectrogram calculations within the MFCC pipeline. The grid search focused on two primary variables:

- **Feature Resolution:** The number of Mel filters (`N_MELS`) for spectrograms and the number of coefficients (`N_MFCC`) for MFCCs, with values set to 40 and 80.
- **Temporal Step (HOP_LENGTH):** Values of 160 and 320 samples. A smaller hop length (160) results in a higher density of features over time.

Training Setup To evaluate the stability of the training process and ensure statistical reliability, each of the eight configurations was trained 3 times using different random seeds. In these experiments, we employed a volume of “silence” samples equivalent to the population of a single core target class. Additionally, 80% of the total available recordings from the “unknown” class were utilized during the training process.

4.2 Transformer parameters experiment

In these experiments, we analyze the impact of key architectural hyperparameters of the Transformer encoder within the AST model on classification performance. In particular, we focus on the embedding dimensionality, the number of attention heads, and the patch embedding resolution, all of which directly influence the model’s representational capacity and the granularity of the input tokenization.

Common setup Across all configurations, the training procedure and data processing pipeline remain identical to ensure comparability. The models are trained using the MFCC-based input representation with fixed preprocessing parameters (`N_MFCC` = 40, `HOP_LENGTH` = 160). *SpecAugment* is applied during training with fixed parameters (`freq_mask_param=10`, `time_mask_param=8`). Each configuration is trained using three different random seeds to account for training variability, and early stopping is applied with a patience of 5 epochs.

Explored hyperparameters The experiment evaluates three main architectural components:

- **Embedding dimension (d_{model}):** values of 128, 256, and 384 are tested. This parameter controls the size of the latent representation of each spectrogram patch, directly affecting model capacity and computational cost.

- **Number of attention heads:** values of 2, 4, and 8 are evaluated (with the constraint that d_{model} is divisible by the number of heads). This parameter determines the level of parallelization in the multi-head self-attention mechanism and the diversity of learned feature subspaces.
- **Patch size:** configurations of (8×8) , (16×16) , (8×16) , and (16×8) are investigated. The patch size controls the spatial granularity of the input tokenization, influencing the trade-off between local detail and sequence length.

Training protocol Each configuration is trained for 30 epochs using the Adam optimizer with a learning rate of 10^{-4} . To ensure statistical robustness, every experiment is repeated for three independent runs with different random seeds. The best-performing model according to validation accuracy is used for further evaluation. In all experiments, class balancing is maintained using a fixed ratio of silence and unknown samples, consistent with the setup used in the input representation experiments.

4.3 Analysis of under/oversampling strategies for silence/unknown classes

In this experiment, we investigate the impact of training set balancing on the performance of the Mini-Conformer model. The primary focus is on identifying the optimal proportions for non-target classes:

- **Silence:** Background noise extracted from ambient environment recordings.
- **Unknown:** Speech commands that are not part of the 10 core target words.

Common Features A grid search consisting of 25 distinct configurations was conducted by varying the UNKNOWN_RATIOS and SILENCE_RATIOS within the following set: $\{0.05, 0.1, 0.5, 1.0, 2.0\}$. It is important to clarify the calculation methodology: these ratios are defined relative to the **total number** of samples across all core target classes. Consequently, a ratio of 0.1 (10%) indicates that the sample size for the respective class is equivalent to the average size of a single core target class (assuming 10 primary commands). A ratio of 1.0 (100%) represents a scenario where the “unknown” or “silence” population equals the combined total of all core classes.

Training Setup Due to the extensive scale of the parameter grid and significant computational constraints, each of the 25 neural networks was trained only once. Training was conducted for a maximum of 30 epochs, utilizing an **Early Stopping** mechanism with a patience of 5.

- **Silence Generation:** We implemented a random cropping technique, extracting 1-second segments with random offsets from the audio files located in the `__background_noise__` directory.
- **Unknown Generation:** We performed sampling with replacement from all available recordings belonging to non-target categories.

4.4 Analysis of single-head and double-head AST architectures

In this experiment, we conduct an analysis of two variations of the Audio Spectrogram Transformer architecture:

- **Baseline AST:** A standard transformer implementation utilizing a single classification head for 12-class recognition.
- **Double-head AST:** A modified version incorporating an auxiliary *Silence Head* for binary “Voice” vs. “Silence” classification. The primary objective is to determine if this hierarchical structure assists in better balanced accuracy performance.

Common Features To ensure a fair comparison, both architectures share identical encoder hyperparameters: an embedding dimension `embed_dim=256`, 8 attention heads (`num_heads`), and a transformer depth of 6 blocks. The input size is set to $(40, 101)$. Patching is performed with a `patch_size` of $(16, 16)$, a `stride` of $(10, 10)$, and a `dropout` rate of 0.1.

Training Setup To guarantee statistical reliability, each model was trained three times using different random seeds. The training was limited to a maximum of 50 epochs, incorporating an **Early Stopping** mechanism with a patience of 5. Based on the results of previous balancing experiments, we fixed the ratios at `silence_ratio` = 0.1 and `unknown_ratio` = 2.0, as this configuration demonstrated good performance.

4.5 Different architecture comparison

In this experiment, we compare the best-performing configurations of each investigated architecture in order to evaluate their relative effectiveness for speech command recognition. The comparison includes four models: the AST-based transformer, the MiniConformer, the pretrained Wav2Vec2 model, and the proposed hierarchical AST variant. Each model is evaluated using its optimal hyperparameters identified in previous experiments, ensuring a fair comparison across architectures.

Common setup All models are trained and evaluated under consistent experimental conditions to ensure comparability. The same dataset split and class configuration are used across all experiments, including 12 output classes (10 commands, “unknown”, and “silence”). For spectrogram-based models (AST and Conformer), MFCC representations with fixed parameters (`N_MFCC` = 40, `HOP_LENGTH` = 160) are used. The AST models operate on MFCC inputs, while the Wav2Vec2 model is trained directly on raw waveform input. Data augmentation is applied using *SpecAugment* for spectrogram-based models, while early stopping is used for all architectures to prevent overfitting.

Evaluated architectures The following configurations are selected as the best-performing variants from previous experiments:

- **AST:** Transformer-based spectrogram model using patch-based tokenization with a patch size of (16×16) , embedding dimension of 256, and 8 attention heads. The model is trained on MFCC input representations and uses a standard classification token for prediction.
- **MiniConformer:** Hybrid architecture combining self-attention and convolutional modules. The selected configuration uses an embedding dimension of 128, 4 attention heads, and a kernel size of 7, trained on MFCC features with patch-based input encoding.
- **Wav2Vec2:** Pretrained self-supervised speech representation model fine-tuned for the classification task. The feature extractor is frozen, while higher encoder layers are partially fine-tuned. The model operates directly on raw waveform input sampled at 16 kHz.
- **Hierarchical AST:** A modified AST architecture with a dual-head classification strategy. One head is responsible for distinguishing speech from silence, while the second head performs command classification over 11 categories. The model uses the same patch embedding configuration as the standard AST.

Training protocol All Transformer-based models are trained for 30 epochs using the Adam optimizer with a learning rate of 10^{-4} . The Wav2Vec2 model is trained for 10 epochs due to its significantly higher computational cost and faster convergence during fine-tuning. Each experiment is repeated using a fixed random seed to ensure reproducibility, and early stopping with a patience of 5 epochs is applied in all cases.

Evaluation objective The goal of this comparison is to analyze how different architectural inductive biases - self-attention over spectrogram patches (AST), convolution-augmented attention (Conformer), hierarchical classification structure (Hierarchical AST), and self-supervised waveform representations (Wav2Vec2) - affect performance in keyword spotting under identical data conditions.

5 Results

5.1 Input representation experiment

Table 1: MiniConformer performance using Mel-spectrogram features.

Configuration	Best Accuracy (%)	Best Balanced Accuracy (%)
$N_{MELS} = 40, H = 160$	94.45 ± 0.26	$93.19 \pm \text{N/A}$
$N_{MELS} = 40, H = 320$	91.15 ± 0.32	89.75 ± 0.30
$N_{MELS} = 80, H = 160$	92.96 ± 0.22	92.19 ± 0.29
$N_{MELS} = 80, H = 320$	92.10 ± 0.24	90.60 ± 0.04

Table 2: MiniConformer performance using MFCC features. Values marked as “N/A” indicate that standard deviation could not be calculated because only one run reached the best epoch before being terminated by the early stopping mechanism.

Configuration	Best Accuracy (%)	Best Balanced Accuracy (%)
$N_{MFCC} = 40, H = 160$	94.88 ± 0.12	93.44 ± 0.12
$N_{MFCC} = 40, H = 320$	$92.97 \pm \text{N/A}$	$89.79 \pm \text{N/A}$
$N_{MFCC} = 80, H = 160$	94.62 ± 0.26	93.00 ± 0.29
$N_{MFCC} = 80, H = 320$	$92.61 \pm \text{N/A}$	$89.76 \pm \text{N/A}$

The results of the input representation experiment are summarized in Tables 1- 2 and Figures 1-2 , with full training curves and confusion matrices provided in Appendix A (Figures 11–14).

Hop length proves to be the most influential hyperparameter across both feature types. Configurations with a smaller hop length ($H = 160$) consistently outperform those with $H = 320$ by a substantial margin – roughly 2-4 percentage points in balanced accuracy – regardless of whether Mel-spectrograms or MFCCs are used. This is expected: a denser temporal sampling captures rapid phonetic transitions more faithfully, which is critical for distinguishing short commands.

Feature type (Mel-spectrogram vs. MFCC) has a comparatively minor effect. The best MFCC configuration ($N_{MFCC} = 40, H = 160$) achieves 94.88% accuracy and 93.44% balanced accuracy, marginally outperforming the best Mel-spectrogram configuration ($N_{MELS} = 40, H = 160$) at 94.45% and 93.19% respectively. The differences fall within the standard deviation ranges, so neither representation can be declared a clear winner based on these results alone.

Feature resolution (N_{MELS} or N_{MFCC} equal to 40 vs. 80) exhibits a somewhat counterintuitive pattern: the lower-resolution setting (40) performs on par with or better than 80 in both feature types, particularly at $H = 160$. This suggests that for 1-second, 16 kHz clips, 40 coefficients already capture the phonetically relevant information, while doubling the resolution primarily adds noise and increases input dimensionality without a measurable benefit.

The confusion matrices (Figures 13 and 14) reveal that the most frequent error sources are consistent across configurations: e.g. command *go* shows high misclassification rates with *no* – likely due to phonetic similarity. In general, error analysis reveals that the primary source of performance degradation across most configurations is the misclassification of core target commands as the “unknown” category. This suggests that the model occasionally exhibits an overly conservative bias in identifying specific speech patterns.

However, the configuration using $N_{MFCC} = 40$ and $H = 160$ demonstrates the most robust performance, significantly reducing such omissions compared to other settings. Due to these, this configuration is selected to be used for all subsequent experiments.

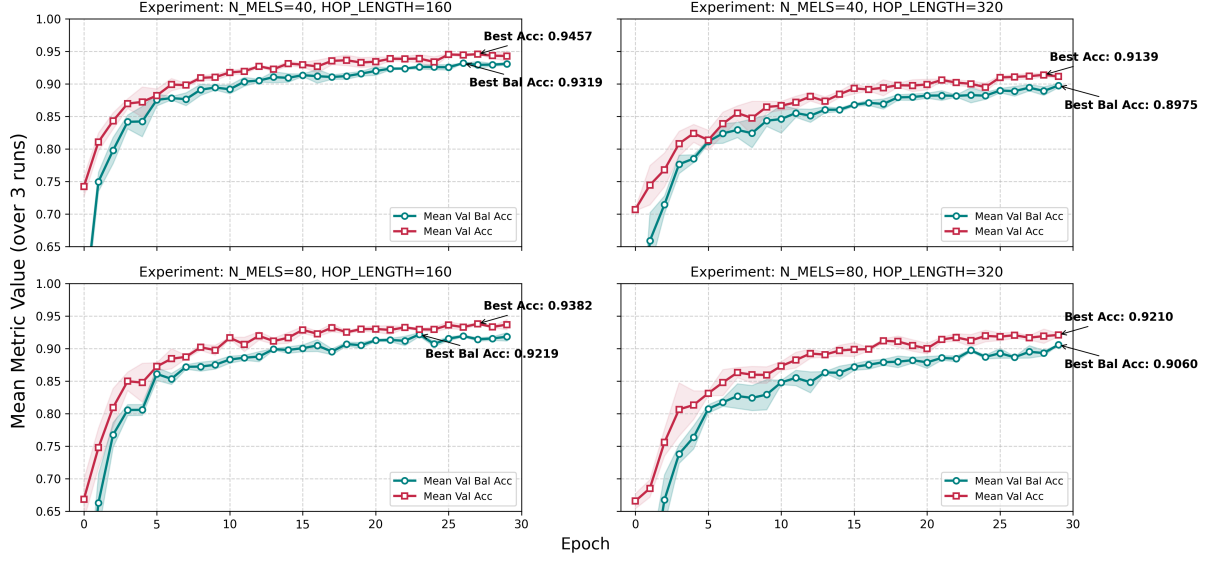


Figure 1: Validation Accuracy and Balanced Accuracy during training across different Mel-spectrogram configurations. Averaged over 3 runs.

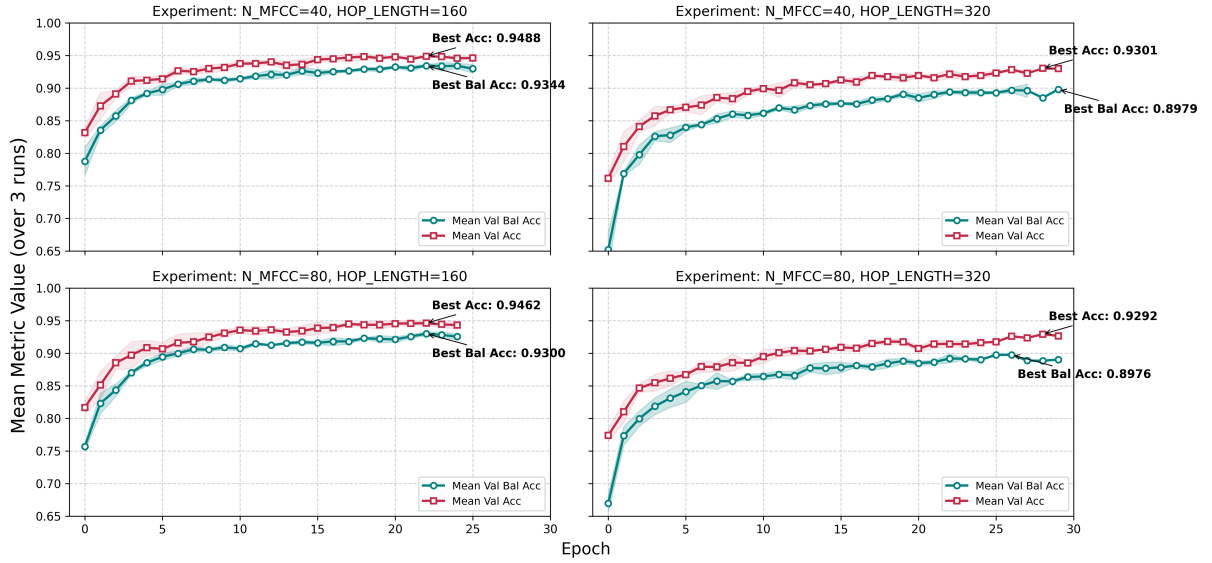


Figure 2: Validation Accuracy and Balanced Accuracy during training across different MFCC configurations. Averaged over 3 runs.

5.2 Transformer parameters experiment

The results of the Transformer architecture experiment are summarized in Figures 3–5.

Patch size emerges as a key factor influencing performance, though not in the initially expected direction. Contrary to the assumption that finer-grained patches would better capture local spectro-temporal structure, the *largest* patch configuration (16×16) achieves the best overall performance, reaching approximately 95.8% accuracy and about 93.5% balanced accuracy. Smaller or asymmetric patches (e.g. 8×8 , 8×16 , 16×8) consistently perform slightly worse, typically by 1–2 percentage points in accuracy.

This suggests that, for short 1-second commands, a coarser tokenization is sufficient and may even be beneficial. Larger patches reduce sequence length, simplifying the attention mechanism and enabling the

model to focus on more global patterns rather than fine-grained local variations, which may otherwise introduce redundancy.

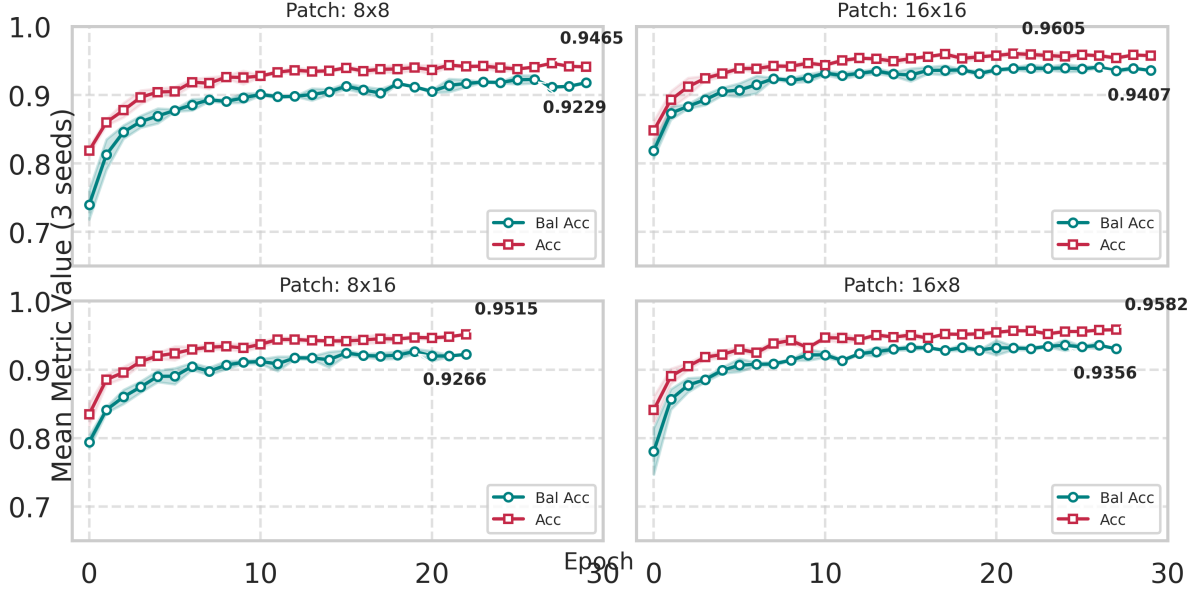


Figure 3: Validation Accuracy and Balanced Accuracy during training across different patch sizes. Averaged over 3 runs.

Embedding dimension (d_{model}) shows a clear but saturating effect. Increasing the dimensionality from 128 to 256 leads to a substantial improvement in both accuracy and balanced accuracy. The best configuration (typically $d_{\text{model}} = 256$ or 384) reaches around 96.0% accuracy and approximately 94.0% balanced accuracy. However, the gain from 256 to 384 is relatively small and often within the standard deviation, indicating diminishing returns as model capacity increases.

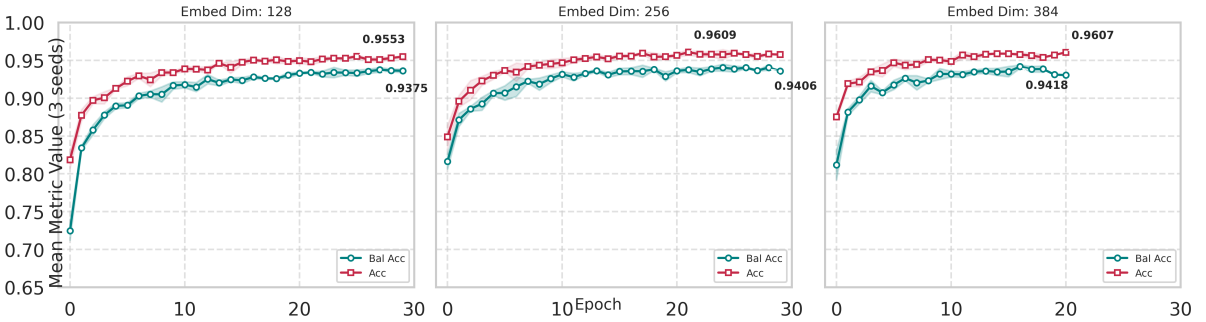


Figure 4: Validation Accuracy and Balanced Accuracy during training across different embedding dimensions. Averaged over 3 runs.

Number of attention heads has the least pronounced impact among the tested parameters. All configurations (2, 4, and 8 heads) converge to similar performance levels, with peak accuracy around 95.5%–96.0% and balanced accuracy near 94%. While higher head counts (4 or 8) tend to produce slightly more stable training dynamics, the differences remain minor and inconsistent across runs.

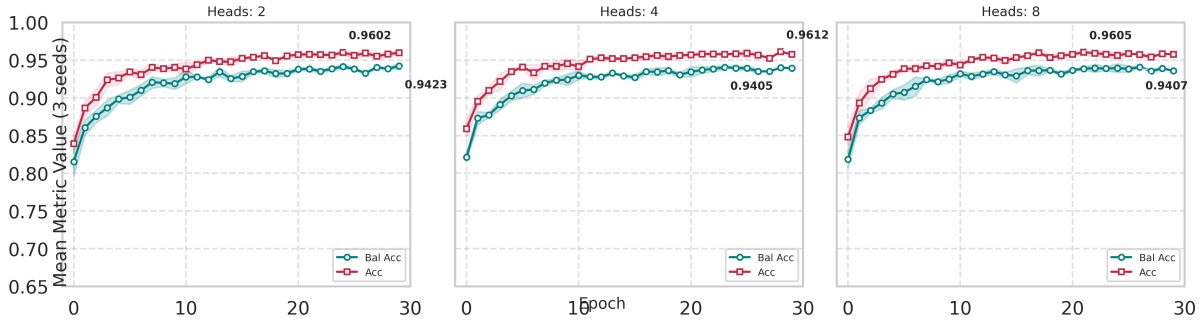


Figure 5: Validation Accuracy and Balanced Accuracy during training across different number of heads. Averaged over 3 runs.

Overall, the experiment indicates that increasing model capacity (via embedding dimension) and reducing sequence length (via larger patch sizes) are more beneficial than increasing attention parallelism. The best-performing configuration combines a 16×16 patch size with a moderate-to-high embedding dimension ($d_{\text{model}} = 256$ or 384), while the number of attention heads plays a secondary role. This configuration is therefore selected for subsequent experiments.

5.3 Analysis of under/oversampling strategies for silence/unknown classes

The results of the 5×5 grid search of data ratios are presented in Figures 6–8, with training and validation loss curves provided in Appendix B (Figure 15).

Impact of the unknown ratio. Increasing the number of *unknown* samples has a strong and consistent positive effect on model performance. Reading Figure 6 row by row, the maximum balanced accuracy rises from roughly 91–93% at **U-Ratio** = 0.05 to 93–94% at **U-Ratio** = 2.0, regardless of the silence ratio.

The F1-score bar charts in Figure 7 make this trend even more explicit: the mean F1 for core commands increases monotonically with the unknown ratio – from approximately 77% at the lowest setting to over 92% at the highest – while the F1 for *unknown* itself grows from $\sim 85\%$ to $\sim 97\%$, indicating that the model learns a much tighter decision boundary for this class as more training examples become available. The stability of F1 within core commands also grows with the number of training “unknown” samples.

The confusion matrices in Figure 8 confirm the mechanism: at **U-Ratio** = 0.05 (left matrix), a large number of core-command samples is misclassified as *unknown* – the horizontal *unknown* column shows substantial off diagonal mass for nearly every command. At **U-Ratio** = 2.0 (right matrix), this horizontal leakage is dramatically reduced; the dominant remaining errors shift to the vertical direction, i.e. a small fraction of *unknown* utterances is now attributed to core commands, which is a considerably less harmful error type for a keyword-spotting system.

Impact of the silence ratio. By contrast, varying the number of *silence* samples has a negligible effect across the board. Scanning any column of Figure 6, the balanced accuracy values fluctuate by at most 1 percentage point with no consistent trend. The F1 scores for the *silence* class are near-perfect ($\geq 99.5\%$) at every configuration, confirming that the class is acoustically well-separated from speech and requires only a minimal number of training examples. Increasing the silence ratio therefore provides no measurable benefit and only inflates training time.

The results indicate that the *unknown* class should be oversampled as aggressively as computationally feasible. The *silence* ratio, on the other hand, can be kept small without any performance penalty. Taking this into consideration we consider **S-Ratio** = 0.1 and **U-Ratio** = 2.0 as a feasible one to use.

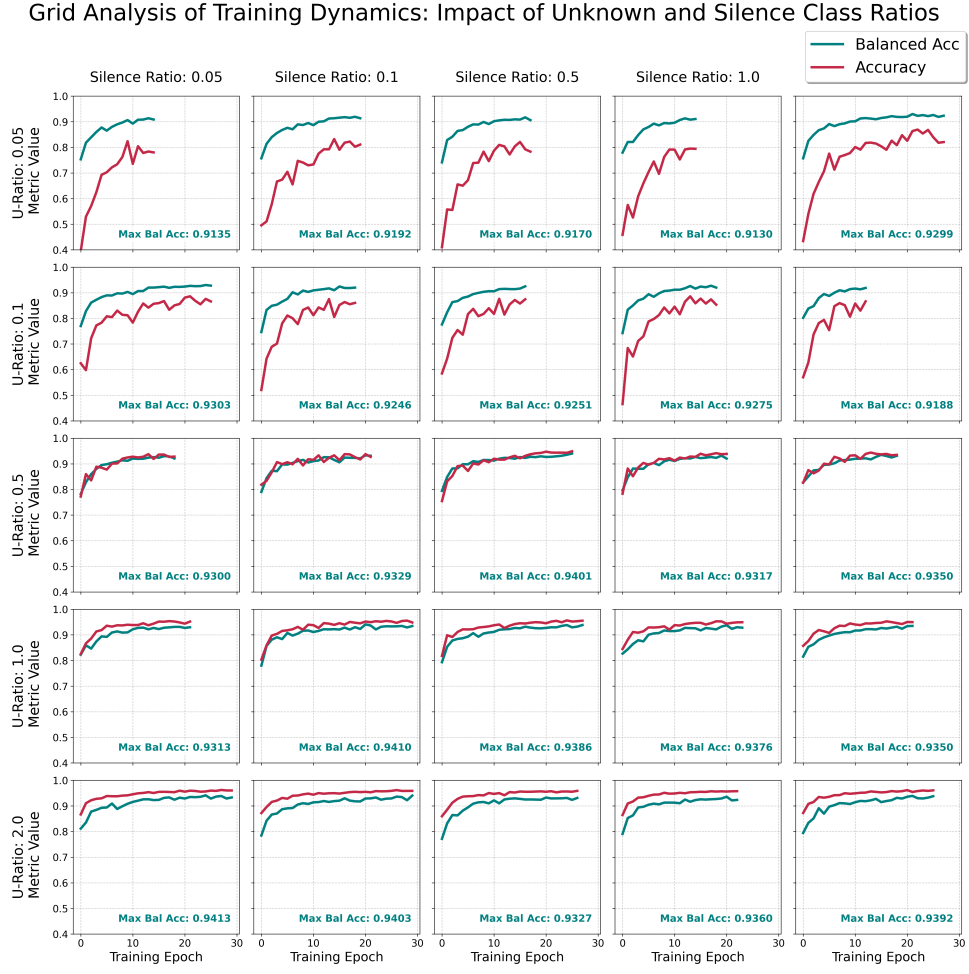


Figure 6: Accuracy and balanced accuracy during training across all 25 (U-Ratio, S-Ratio) configurations.

F1 Analysis: Core (Mean $\pm$ Std) vs Non-target Classes

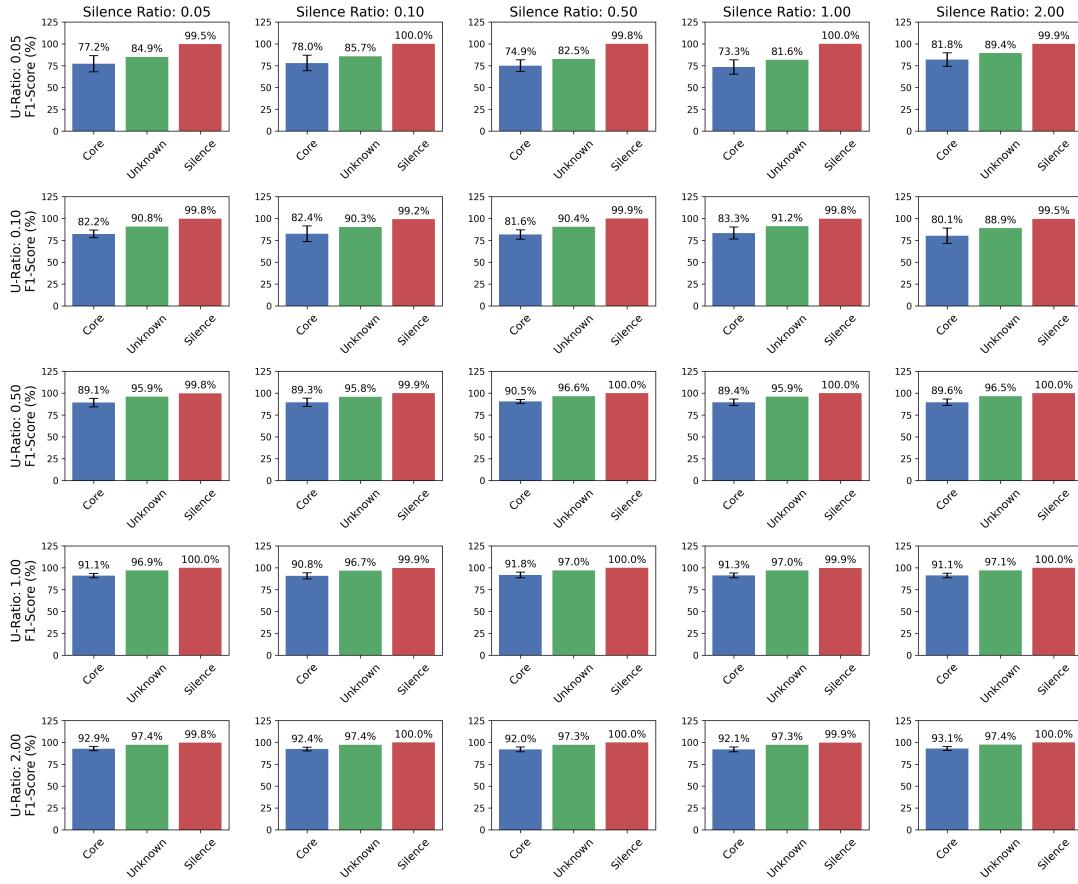


Figure 7: Per-group F1 scores (Core, Unknown, Silence) across all 25 (U-Ratio, S-Ratio) configurations.

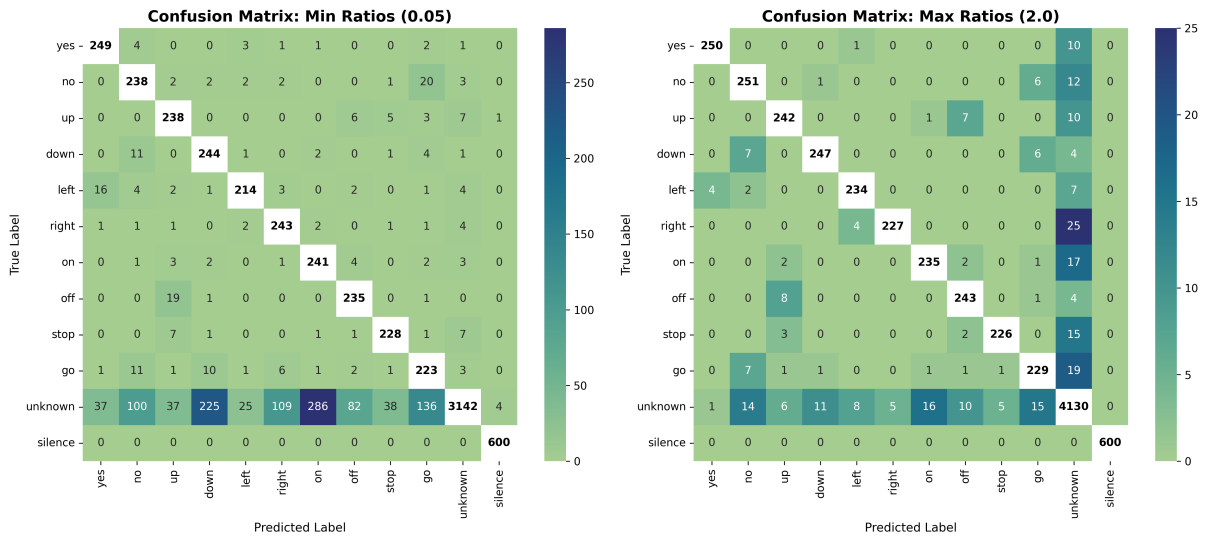


Figure 8: Confusion matrices on validation set for one of the worst (U-Ratio = 0.05, S-Ratio = 0.05) and best (U-Ratio = 2.0, S-Ratio = 2.0) configurations.

5.4 Analysis of single-head and double-head AST architectures

The results are summarized in Table 3, with training curves in Figure 9 and per-class precision and recall matrices in Appendix C (Figures 16–18).

The Double-head AST consistently outperforms the baseline on both metrics: balanced accuracy improves from $93.27 \pm 0.12\%$ to $94.03 \pm 0.31\%$, and overall accuracy from $96.16 \pm 0.17\%$ to $96.58 \pm 0.24\%$. The improvement is achieved at the cost of slower convergence (19.7 ± 7.3 epochs vs. 14.0 ± 1.6 for the baseline), reflecting the additional complexity of jointly optimizing two heads.

The precision and recall matrices in Appendix C reveal that the gain is primarily driven by improved recognition of core commands and the *unknown* class: the auxiliary silence head frees the primary head from having to model the silence boundary, allowing it to better discriminate between the 11 remaining classes. Overall, the dual-head architecture offers a consistent improvement with no changes to the encoder or training data, and is therefore the preferred design.

Table 3: Performance Comparison: Baseline AST vs. Multi-head AST (Mean $\pm$ Std Dev). Averaged over 3 runs

Model	Accuracy (%)	Balanced Accuracy (%)	Epochs to Best
Baseline AST	96.16 ± 0.17	93.27 ± 0.12	14.0 ± 1.6
Double-head AST	96.58 ± 0.24	94.03 ± 0.31	19.7 ± 7.3

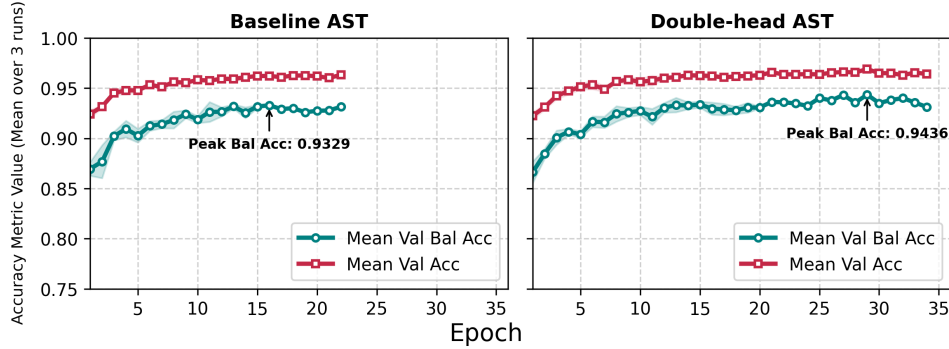


Figure 9: Validation accuracy and balanced accuracy during training for Baseline AST and Double-head AST, averaged over 3 runs.

5.5 Different architecture experiment

In this experiment, we compare the training dynamics and final performance of four different architectures: AST, Conformer, Wav2Vec-based model, and Hierarchical AST. The evolution of training and validation loss, as well as accuracy and balanced accuracy, is summarized in Figure 10, with training and validation loss curves provided in Appendix D (Figure 19).

Overall learning behaviour reveals clear differences in convergence speed and, more importantly, in *balanced validation accuracy*, which is the primary evaluation metric due to class imbalance in the dataset. The AST and Conformer models show smooth and stable optimization trajectories, with steadily decreasing training loss and gradual improvements in balanced validation accuracy across epochs. The Wav2Vec-based model exhibits a less stable learning pattern, with noticeable fluctuations in validation performance, including balanced accuracy, indicating higher sensitivity to training dynamics in this setup. In contrast, the Hierarchical AST demonstrates the most stable and consistent behaviour, achieving steady improvements and low variance in balanced validation accuracy throughout training.

AST vs Conformer. Both models achieve strong and comparable performance in terms of balanced validation accuracy, which stabilizes around 0.92–0.94 in later epochs. AST converges slightly faster and reaches marginally higher peak balanced accuracy, while also achieving slightly lower validation loss (approximately 0.14–0.15 compared to 0.16–0.17 for Conformer). However, these differences remain small and within the range of run-to-run variability, indicating that both architectures provide similarly strong performance for this task.

Wav2Vec-based model achieves competitive peak performance in terms of balanced validation accuracy, reaching values close to 0.95–0.96 in the best runs. However, this performance is less stable

across epochs and runs, with higher variance in balanced accuracy compared to AST- and Conformer-based models. This suggests that, despite strong representational power, the model is more sensitive to optimization conditions in this training setup.

Hierarchical AST consistently achieves the best performance across all architectures in terms of balanced validation accuracy. It reaches the highest and most stable values, exceeding 0.94 and peaking around 0.94–0.94+ in later epochs, while maintaining low variance across runs. This improvement is accompanied by the lowest validation loss (approximately 0.12–0.13), indicating both better generalization and more confident predictions. Importantly, it maintains this performance consistently throughout training, rather than only at isolated epochs.

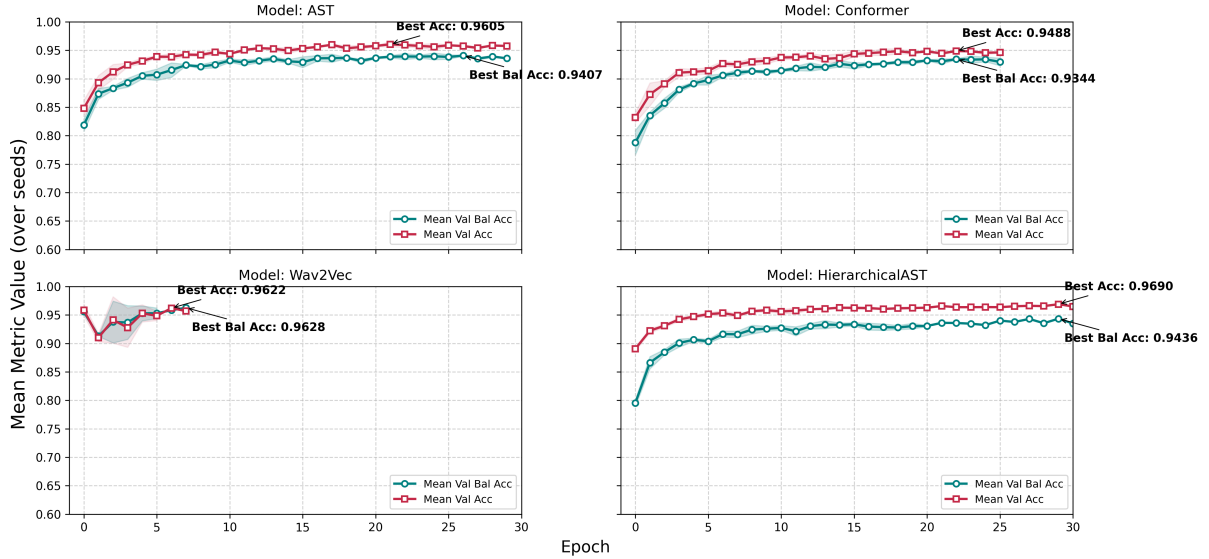


Figure 10: Validation accuracy and balanced accuracy during training for different architectures, averaged over 3 runs.

Overall, the results indicate that Hierarchical AST provides the strongest inductive bias for this task, leading to the highest and most stable balanced validation accuracy. While AST and Conformer remain strong and competitive baselines, and Wav2Vec achieves high peak performance but lower stability, the Hierarchical AST offers the best trade-off between accuracy, robustness, and class-balanced performance.

6 Conclusion

This study presented a comprehensive empirical evaluation of attention-based architectures for keyword spotting on the TensorFlow Speech Commands dataset, with a particular focus on transformer variants and modern speech representation techniques. Across all experiments, performance was assessed primarily using balanced accuracy to account for class imbalance, supported by confusion matrix analysis to better understand systematic error patterns.

The results highlight several consistent findings regarding input representations and model design choices. First, for time–frequency representations, both MFCC and Mel-spectrogram features yield highly competitive results when used with lightweight transformer encoders such as the MiniConformer. Differences between these representations are marginal, indicating that, for short utterances, performance is more strongly influenced by temporal resolution than by the exact spectral representation. In contrast, the hop length was found to be a critical hyperparameter, with denser temporal sampling ($H = 160$) consistently improving performance by enabling better modelling of short phonetic transitions. Interestingly, increasing feature resolution beyond a moderate number of coefficients provided no meaningful benefit, suggesting that compact representations are sufficient for this task.

For transformer-based architectures, architectural capacity and tokenization strategy proved more influential than fine-grained attention design choices. Larger patch sizes consistently improved performance in the AST model by reducing sequence length and encouraging more global reasoning over the spectrogram. Similarly, increasing embedding dimensionality led to clear performance gains, though with

diminishing returns beyond a certain capacity. In contrast, the number of attention heads had only a minor effect, indicating that parallel attention mechanisms are less critical than overall representation capacity and sequence efficiency in this setting.

The analysis of data balancing strategies further emphasized the importance of dataset composition. Oversampling the *unknown* class significantly improved both overall and per-class performance, reducing the tendency of models to misclassify valid commands as unknown. In contrast, increasing the proportion of *silence* samples had negligible impact, confirming that this class is already well-separated and requires minimal representation. These findings underline that careful handling of ambiguous or heterogeneous classes is more important than balancing all classes uniformly.

Architectural modifications also demonstrated consistent benefits. The proposed double-head AST improved both accuracy and balanced accuracy by decoupling silence detection from command classification, leading to clearer decision boundaries for the main task. Similarly, the hierarchical AST variant achieved the strongest overall performance across all evaluated models, combining high accuracy with the lowest variance and most stable training dynamics. While Wav2Vec-based models showed strong peak performance, their higher variability suggests greater sensitivity to training configuration in this setup.

Finally, the cross-architecture comparison confirmed that transformer-based approaches are highly competitive for keyword spotting, with AST and Conformer serving as strong baselines. However, architectures that incorporate task decomposition or hierarchical structure provide the most consistent improvements, particularly under class imbalance conditions.

Overall, the experiments demonstrate that performance in keyword spotting is primarily driven by three factors: (1) appropriate temporal resolution in input representations, (2) efficient architectural design that balances sequence length and model capacity, and (3) effective handling of underrepresented or ambiguous classes, especially the *unknown* category. The best-performing configurations consistently reflect a balance between these aspects rather than reliance on any single architectural component.

Future work should expand the scope of this study in several directions to further strengthen the generality and statistical validity of the findings. First, a broader range of hyperparameters should be systematically explored, including alternative patching strategies, attention mechanisms, and feature extraction settings, to better characterize the sensitivity of transformer-based keyword spotting models. Second, extending the comparison to a wider set of architectures - such as additional hybrid CNN-transformer models, lightweight mobile-oriented networks, and more recent self-supervised speech encoders - would provide a more comprehensive understanding of performance trade-offs across model families. Finally, increasing the number of training runs per configuration would improve the statistical robustness of the reported results, enabling more reliable conclusions about performance differences and reducing the influence of random initialization and training variability. Together, these extensions would help to further clarify which design choices are consistently beneficial across settings and which are more dataset- or implementation-dependent.

7 Instruction of the application (reproducibility)

All experiments presented in this report are fully reproducible using the provided project codebase. The implementation is modular and organized to separate data preprocessing, feature extraction, model definitions, and experiment configurations, allowing each component of the pipeline to be executed independently or as part of full training runs.

7.1 Environment setup

All experiments were implemented in Python (version ≥ 3.12) using the PyTorch ecosystem. The required dependencies are listed below:

```
matplotlib
numpy
pandas
scikit-learn
seaborn
torch
torchaudio
tqdm
```

`transformers`

The project was implemented using the PyTorch framework. Experiments can be executed on a TPU, GPU (CUDA), or CPU. The project automatically selects the best available device in the following order of priority: $\text{TPU} \geq \text{CUDA-enabled GPU} \geq \text{CPU}$. All dependencies are also listed in the `project.toml` file included in the project, which can be used for environment replication.

7.2 Dataset preparation

All experiments use the *Speech Commands Dataset* from the TensorFlow Speech Recognition Challenge [4]. The dataset consists of 1-second audio recordings organized into labeled directories corresponding to spoken commands.

After downloading, the dataset should be placed in the following directory structure without any modifications:

```
root/
|-- tensorflow-speech-recognition-challenge/
|   |-- train/
|       |-- audio/
|           |-- yes/
|           |-- no/
|           |-- up/
|           |-- down/
|           |-- ...
|           |-- _background_noise_/
```

The project includes separate preprocessing scripts for Mel-spectrograms, MFCC features, and raw waveform processing. These scripts must be executed prior to running any training or evaluation experiments, as all models rely on precomputed input representations.

The preprocessing pipeline automatically handles:

- loading raw WAV files,
- resampling audio signals when necessary,
- trimming or padding recordings to a fixed length of 1 second,
- mapping labels into the 12 target classes,
- generating MFCC or Mel-spectrogram representations where required.

7.3 Running experiments

To ensure reproducibility of our experimental results across different execution environments and time-frames, a fixed random seed was set at the start of each experiment. The project includes a dedicated `set_seed()` function that synchronizes the random state across Python, NumPy, and PyTorch, ensuring consistent behavior across runs. Within the experiments directory, each trained model has a subdirectory containing its saved check-points and training history (.csv files). The corresponding .ipynb notebooks can be used to reproduce the training runs, evaluate performance on training, validation, and test sets, generate plots, and implement ensemble strategies. Core modules required by these notebooks are located in the training folder.

8 AI Usage Declaration

AI assistants were used in a supporting role in three areas. First, they assisted with code refactoring and docstring generation to improve code readability. Second, they helped identify original papers in which certain concepts were first introduced (e.g., Curriculum Learning), with all references subsequently verified manually. Third, they were used to correct spelling and grammar in the report to improve the fluency of the written English.

References

- [1] J. Allen. Short term spectral analysis, synthesis, and modification by discrete fourier transform. *IEEE Transactions on Acoustics, Speech, and Signal Processing*, 25:235–238, 1977.
- [2] Alexei Baevski, Henry Zhou, Abdelrahman Mohamed, and Michael Auli. wav2vec 2.0: A framework for self-supervised learning of speech representations. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 33, pages 12449–12460, 2020.
- [3] Steven B. Davis and Paul Mermelstein. Comparison of parametric representations for monosyllabic word recognition in continuously spoken sentences. *IEEE Transactions on Acoustics, Speech, and Signal Processing*, 28(4):357–366, 1980.
- [4] Julia Elliott, Mark McDonald, Pete Warden, and Raziel. Tensorflow speech recognition challenge. <https://kaggle.com/competitions/tensorflow-speech-recognition-challenge>, 2017. Kaggle.
- [5] Yuan Gong, Yu-An Chung, and James Glass. AST: Audio spectrogram transformer. In *Proceedings of Interspeech*, 2021.
- [6] Yuan Gong, Cheng-I Lai, Yu-An Chung, and James Glass. SSAST: Self-supervised audio spectrogram transformer. In *Proceedings of the AAAI Conference on Artificial Intelligence*, pages 10699–10709, 2022.
- [7] Anmol Gulati, James Qin, Chung-Cheng Chiu, Niki Parmar, Yu Zhang, Jiahui Yu, Wei Han, Shibo Wang, Zhengdong Zhang, Yonghui Wu, and Ruoming Pang. Conformer: Convolution-augmented transformer for speech recognition, 2020.
- [8] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 770–778, 2016.
- [9] Geoffrey Hinton, Li Deng, Dong Yu, George E. Dahl, Abdel-rahman Mohamed, Navdeep Jaitly, Andrew Senior, Vincent Vanhoucke, Patrick Nguyen, Tara N. Sainath, and Brian Kingsbury. Deep neural networks for acoustic modeling in speech recognition: The shared views of four research groups. *IEEE Signal Processing Magazine*, 29:82–97, 2012.
- [10] Yann LeCun, Léon Bottou, Yoshua Bengio, and Patrick Haffner. Gradient-based learning applied to document recognition. *Proceedings of the IEEE*, 86(11):2278–2324, 1998.
- [11] Koichi Miyazaki, Tatsuya Komatsu, Tomoki Hayashi, Shinji Watanabe, Tomoki Toda, and Kazuya Takeda. Conformer-based sound event detection with semi-supervised learning and data augmentation. In *Proceedings of the Detection and Classification of Acoustic Scenes and Events (DCASE) Workshop*, 2020.
- [12] Daniel S. Park, William Chan, Yu Zhang, Chung-Cheng Chiu, Barret Zoph, Ekin D. Cubuk, and Quoc V. Le. Specaugment: A simple data augmentation method for automatic speech recognition. In *Interspeech 2019*, page 2613–2617. ISCA, 2019.
- [13] Lawrence Rabiner and Biing-Hwang Juang. *Fundamentals of Speech Recognition*. Prentice-Hall, 1993.
- [14] Roman A. Solovyev, Maxim Vakhrushev, Alexander Radionov, Irina I. Romanova, Aleksandr A. Amerikanov, Vladimir Aliev, and Alexey A. Shvets. Deep learning approaches for understanding simple speech commands. In *2020 IEEE 40th International Conference on Electronics and Nanotechnology (ELNANO)*, pages 688–693, 2020.
- [15] Stanley Smith Stevens, John Volkman, and Edwin B. Newman. A scale for the measurement of the psychological magnitude pitch. *The Journal of the Acoustical Society of America*, 8(3):185–190, 1937.
- [16] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 30, pages 5998–6008, 2017.
- [17] Pete Warden. Speech commands: A dataset for limited-vocabulary speech recognition, 2018.

A Input representation experiment

Figures 11-14 support the analysis of the spectrogram representation of audio files.

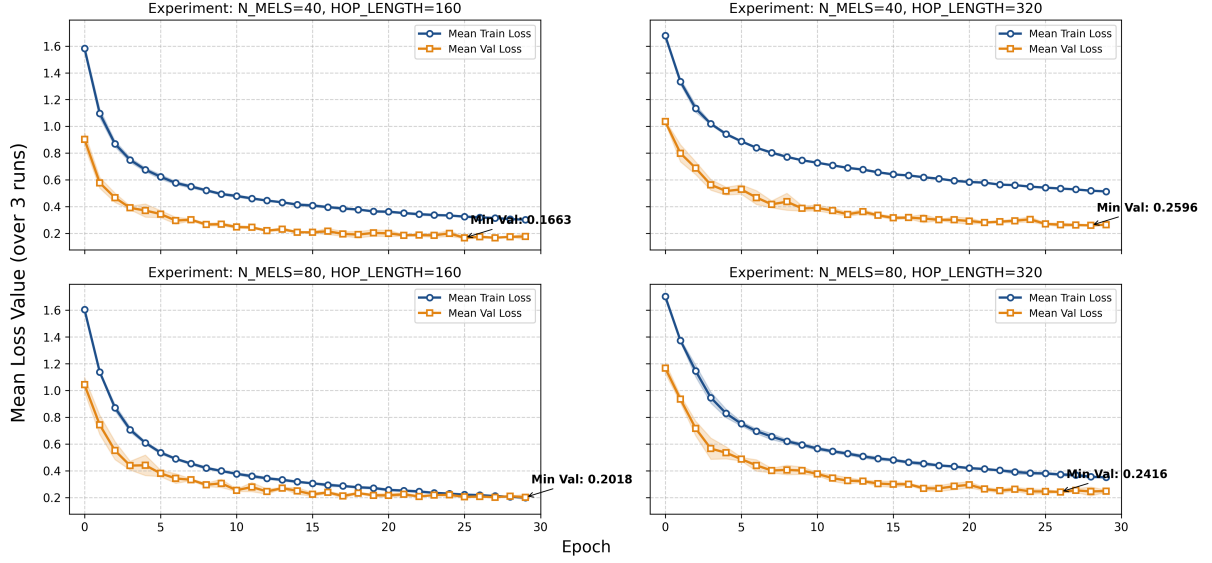


Figure 11: Loss curves (train, validation) across different Mel-spectrogram configurations. Averaged over 3 runs.

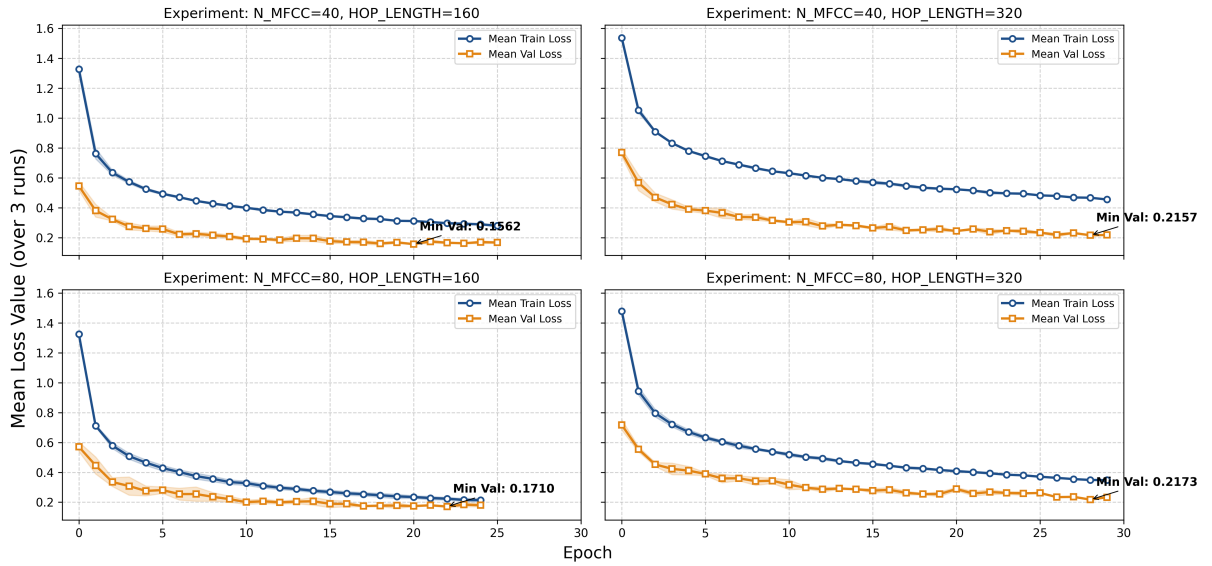


Figure 12: Loss curves (train, validation) across different MFCC configurations. Averaged over 3 runs.

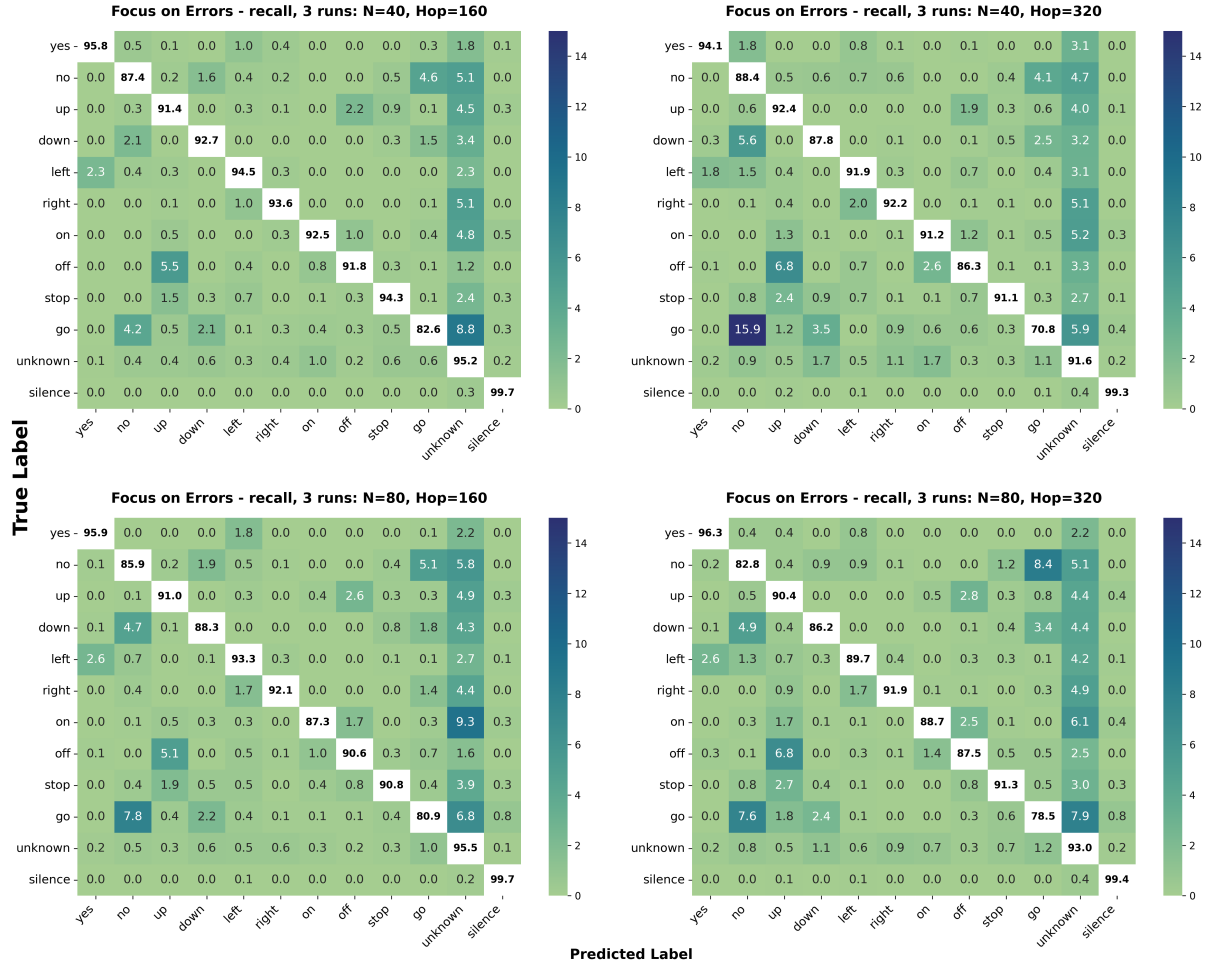


Figure 13: Confusion matrices on validation set for various Mel-spectrogram configurations. The main diagonal is left uncolored, allowing the color gradient to highlight classification errors. The values are row-normalized by the total number of samples per class, representing the Recall metric for each category.

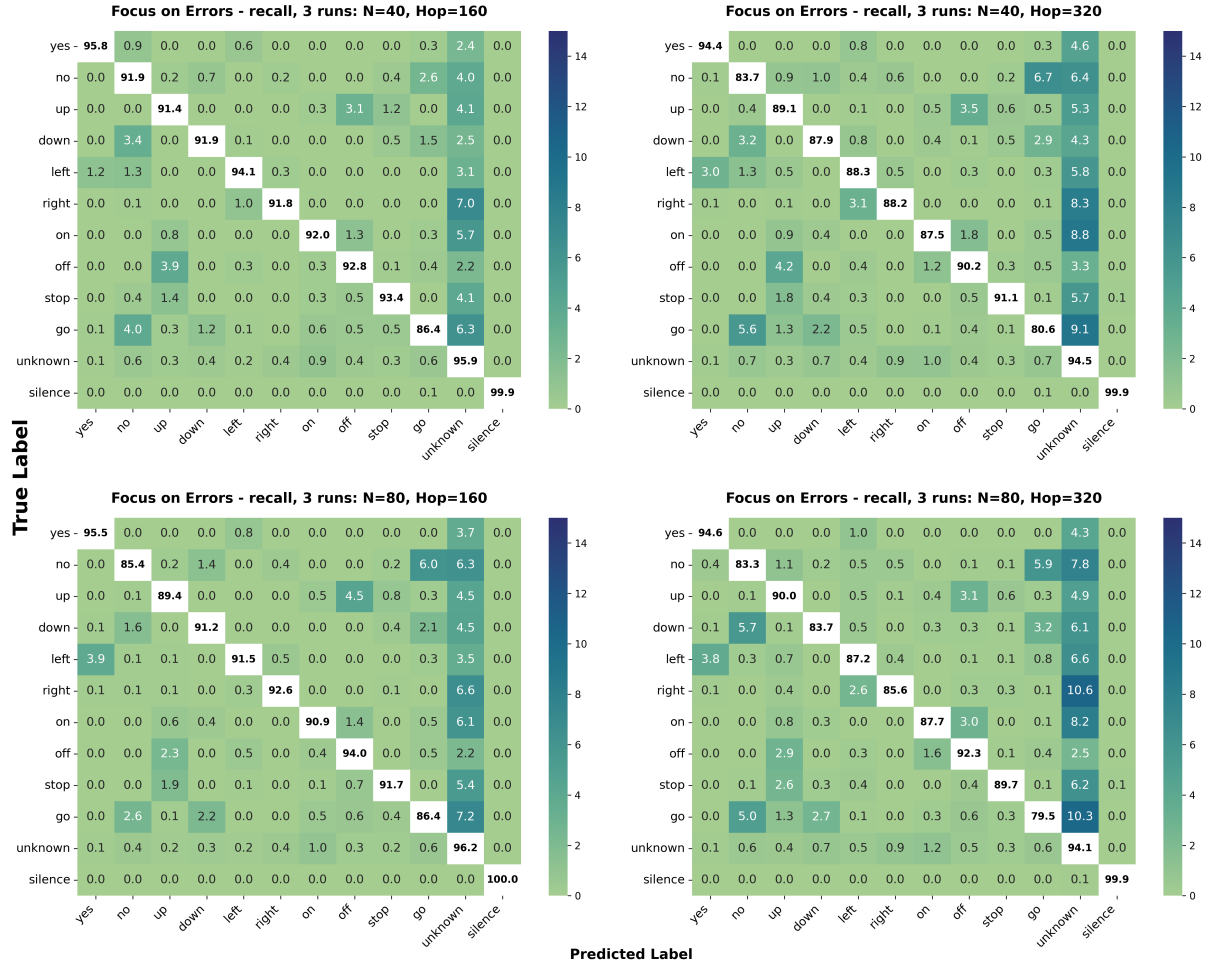


Figure 14: Confusion matrices on validation set for various MFCC configurations. The main diagonal is left uncolored, allowing the color gradient to highlight classification errors. The values are row-normalized by the total number of samples per class, representing the Recall metric for each category.

B Analysis of under/oversampling strategies for silence/unknown classes

Figure 15 supports the analysis of under/oversampling strategies for silence/unknown classes.

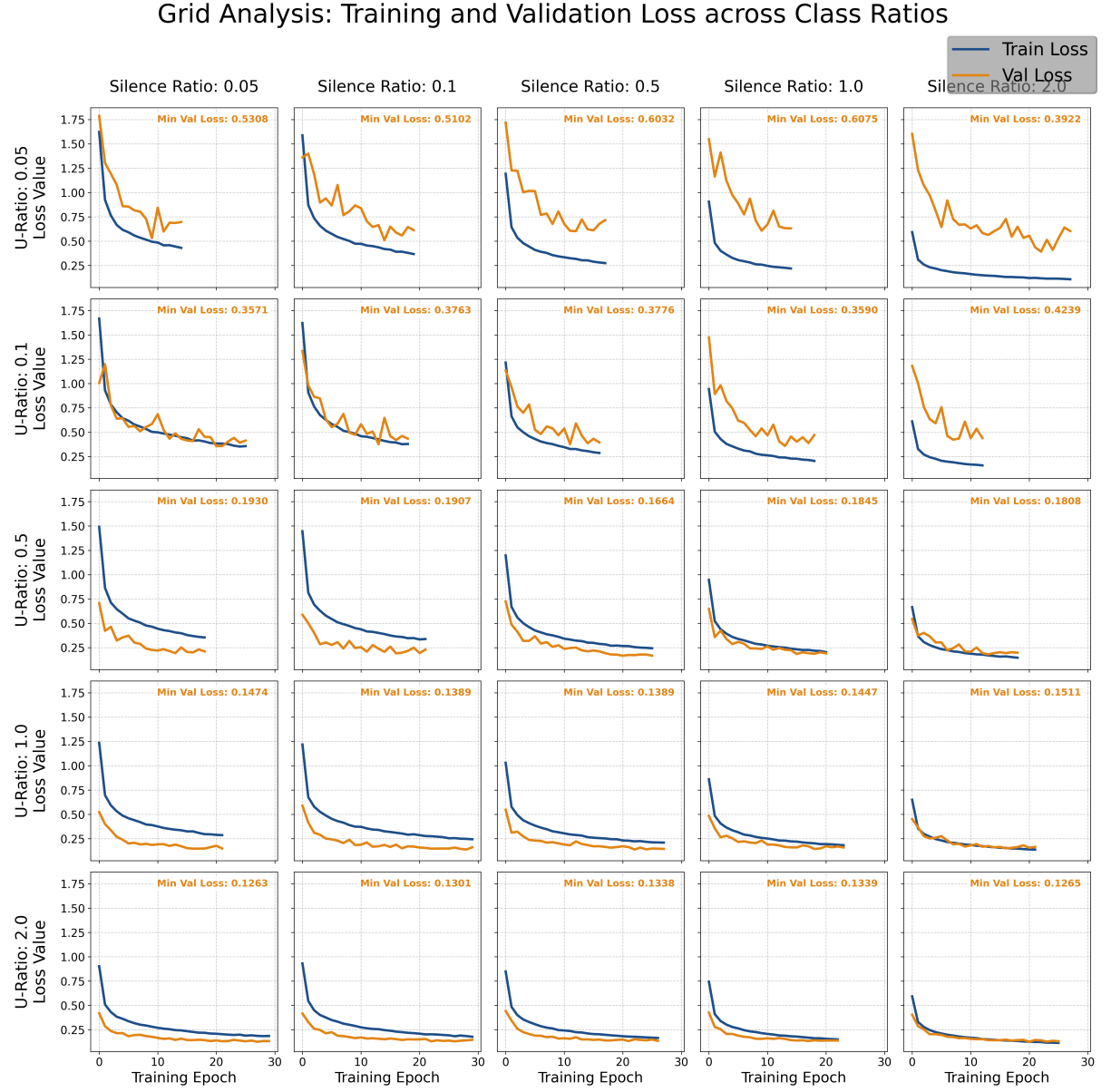


Figure 15: Training and validation loss curves across all 25 (U-Ratio, S-Ratio) configurations.

C Analysis of single-head and double-head AST architectures

Figures 16-18 support the analysis of single-head and double-head AST architectures.

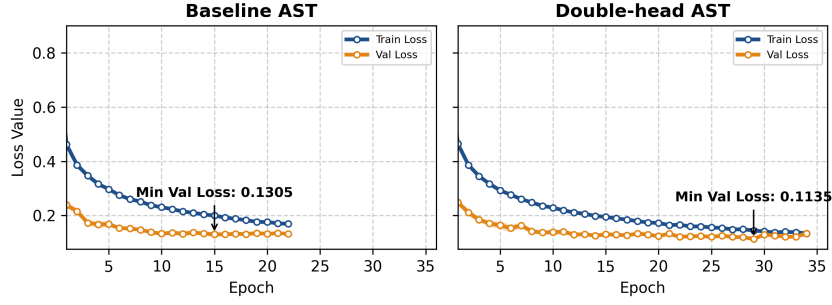


Figure 16: Training and validation loss curves for Baseline AST (left) and Double-head AST (right), averaged over 3 runs.

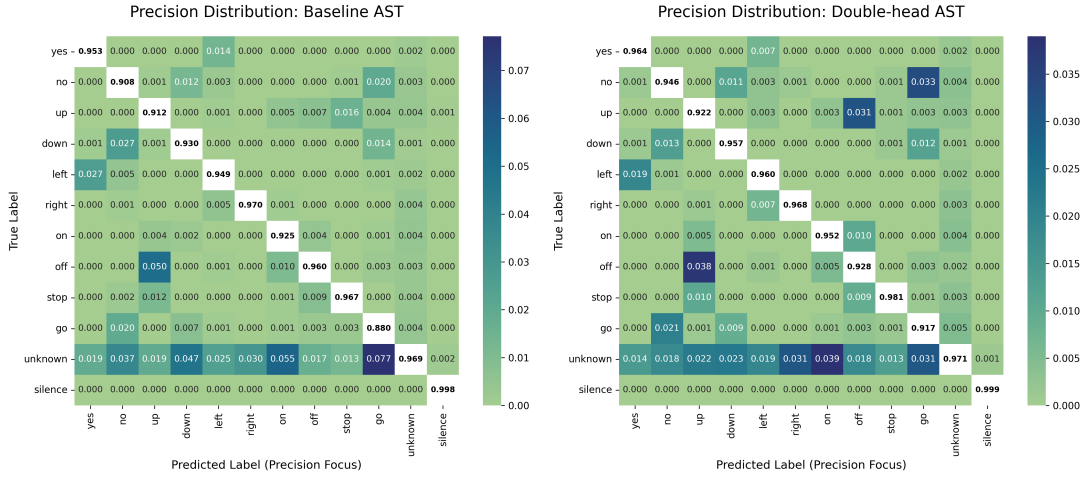


Figure 17: Precision matrices for Baseline AST (left) and Double-head AST (right). Values are normalized column-wise, representing per-class precision.

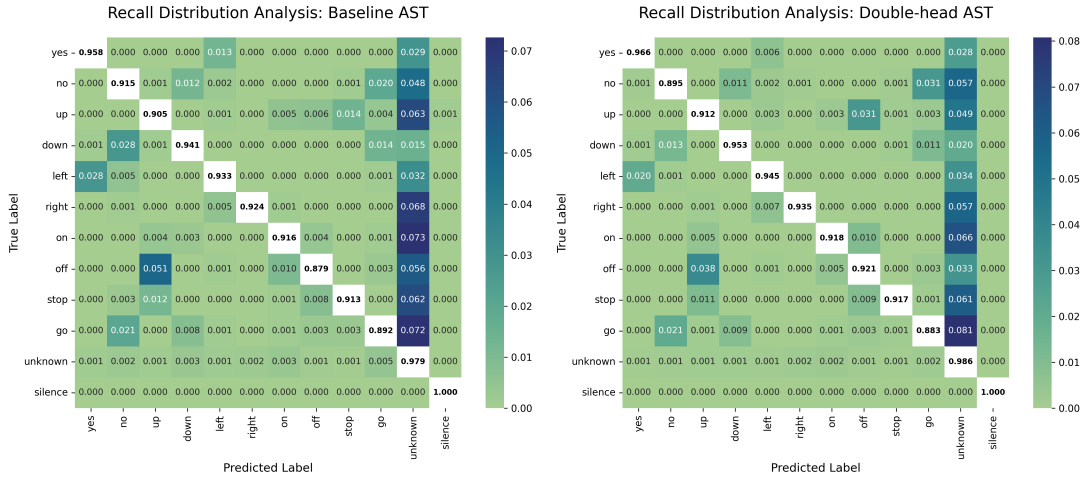


Figure 18: Recall matrices for Baseline AST (left) and Double-head AST (right). Values are normalized row-wise, representing per-class recall.

D Analysis of different model architectures

Figure 19 support the analysis of performance of different model architectures.

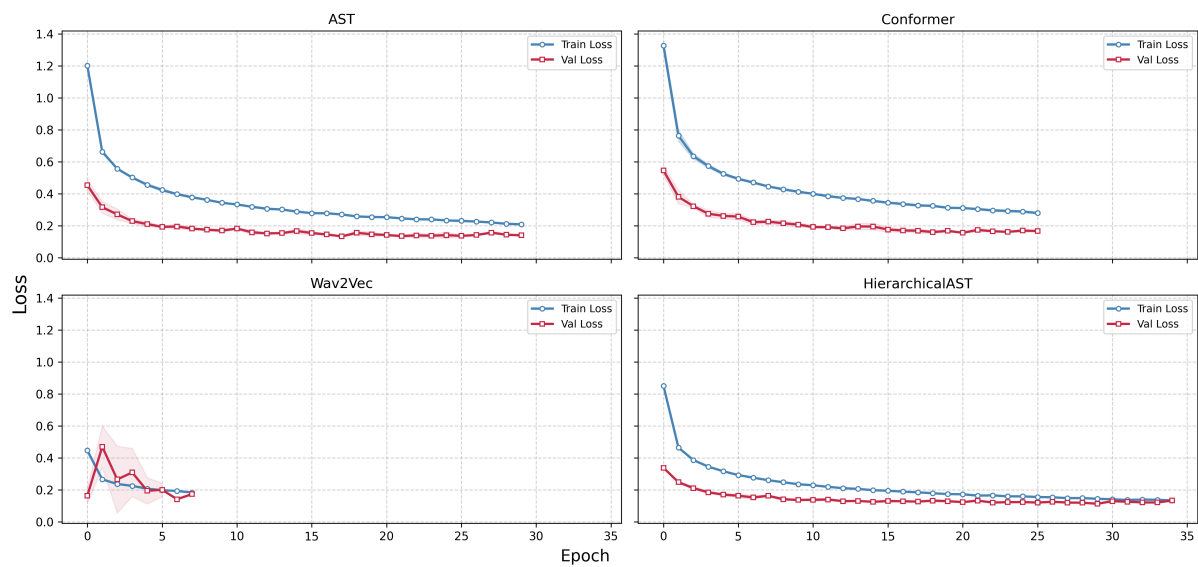


Figure 19: Training and validation loss curves for different architectures, averaged over 3 runs.